

Predicting Urban Mobility Flows with Recurrent Neural Networks: A Grid-Level Spatiotemporal Approach

Introduction

Understanding and forecasting human mobility is a central challenge in urban computing, with direct applications in traffic management, emergency response, and resource allocation. Short-term prediction of population flow—how many people will enter a given area in the next thirty minutes—remains difficult due to the interplay of diurnal rhythms, land-use context, and stochastic behavioral variation. This project investigates next-step mobility flow prediction at the grid-cell level. Using the YJMob100K dataset, I design a spatiotemporal feature representation, train and compare a non-temporal linear baseline against a two-layer LSTM, and use spatial error maps to identify where the model succeeds and where it falls short.

Data and Feature Construction

The dataset used in this project is YJMob100K, introduced in “YJMob100K: City-scale and longitudinal dataset of anonymized human mobility trajectories” (Yabe et al., 2023). YJMob100K is an open, anonymized mobility dataset derived from mobile phone location traces collected by Yahoo Japan / LY Corporation. To protect privacy, space is discretized onto a 500 m $\times$ 500 m grid and time is binned into 30-minute intervals; absolute geographic coordinates and dates are withheld. The dataset also provides a point-of-interest (POI) category count vector of approximately 85 dimensions per grid cell, capturing local urban function.

Feature engineering combines two types of signals. The 85-dimensional POI vector is joined to each observation, providing static land-use context. Temporal position is encoded via sinusoidal functions of time-of-day and day-of-week (`tod_sin`, `tod_cos`, `dow_sin`, `dow_cos`), representing periodicity as smooth cyclical features without arbitrary ordinal assumptions. The resulting per-timestep feature vector encodes what a place is, when it is observed, and its current flow level.

Methodology

The data is split temporally: the first 80% of the time series forms the training set; the remaining 20% (covering 23,302 eligible grid cells) forms the test set. This chronological split simulates real-world deployment and prevents leakage of future information. Normalization statistics are computed exclusively on the training portion.

Sequences are constructed with a sliding window of length 8 (four hours) and stride 4 (two hours), yielding input tensors of shape $[B, 8, F]$. Two architectures are compared. The linear baseline (LinearSeqModel) flattens the window into a single vector and applies one fully connected layer, ignoring temporal order. The LSTM (FlowLSTM) uses two recurrent layers with hidden size 64, reads the sequence step-by-step, and outputs a scalar prediction from the final hidden state. Both models are optimized with Adam ($\text{lr} = 1 \times 10^{-3}$) and MSE loss over eight epochs.

Results and Analysis

The performance gap is large and unambiguous. The linear baseline reaches a test MSE of 533.87 after eight epochs. The LSTM achieves a test MSE of 4.15—a 129-fold improvement (Figure 1). The small train–test gap (train MSE: 3.79 vs. test MSE: 4.15) confirms stable generalization, indicating that the model learns temporal patterns that transfer reliably to future periods.

```
Epoch 08 | Processed 24700 batches so far
Epoch 08 | Processed 24800 batches so far
Epoch 08 | Processed 24900 batches so far
Epoch 08 | Processed 25000 batches so far
Epoch 08 | Processed 25100 batches so far
Epoch 08 | Processed 25200 batches so far
Epoch 08 | Processed 25300 batches so far
Epoch 08 | Processed 25400 batches so far
Epoch 08 | Processed 25500 batches so far
Epoch 08 | Processed 25600 batches so far
Epoch 08 | Processed 25700 batches so far
Epoch 08 | Processed 25800 batches so far
flow_feat_test: 23302 cells with >= 8 records
Epoch 08 | Train MSE 3.7942 | Test MSE 4.1452
Saved checkpoint to /content/drive/MyDrive/MOBILITY/checkpoints_lstm/flow_lstm_epoch08.pth
Saved final model to /content/drive/MyDrive/MOBILITY/flow_lstm_final.pth
```

Figure 1. LSTM training log (Epoch 8). Final train MSE: 3.79, test MSE: 4.15, over 25,800 mini-batches. The linear baseline converges to test MSE 533.87.

Spatial analysis reveals where the LSTM succeeds and fails. Figure 2 shows a snapshot at Day 64, $t = 23$ (~11:30): the predicted map closely mirrors the true flow distribution, reproducing the diffuse low-flow periphery and the concentrated high-flow corridor through the urban core. Prediction errors are highest in the densest central zones, where flows are larger in magnitude and more sensitive to irregular fluctuations.

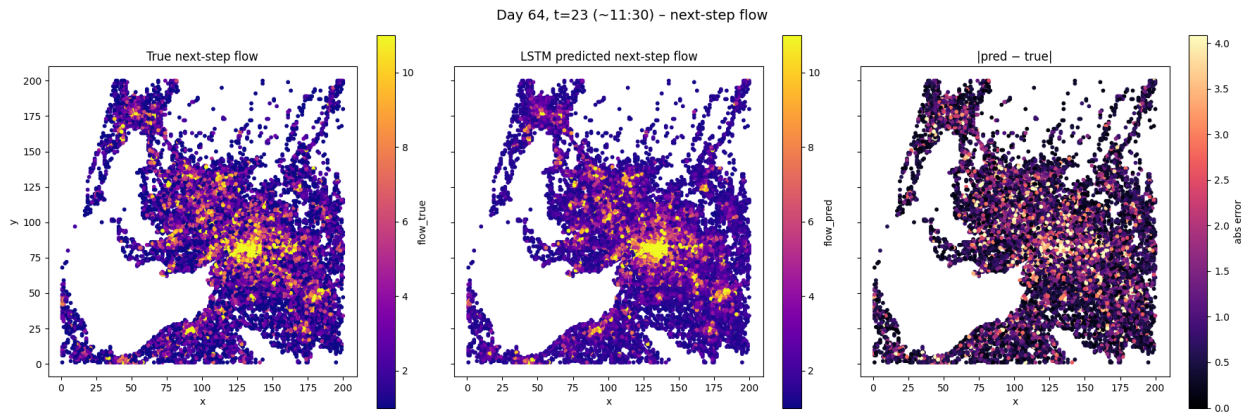


Figure 2. Spatial prediction at Day 64, $t = 23$ (~11:30). Left: true next-step flow; center: LSTM prediction; right: absolute error $|\text{pred} - \text{true}|$.

Figure 3 extends this analysis to six evenly spaced time steps across Day 64. Across all panels, the spatial error pattern remains consistent: peripheral areas are predicted reliably while the urban core concentrates residuals. The model tracks the evolving intensity of mobility from morning through late evening, suggesting it has internalized the regular temporal structure of urban movement.

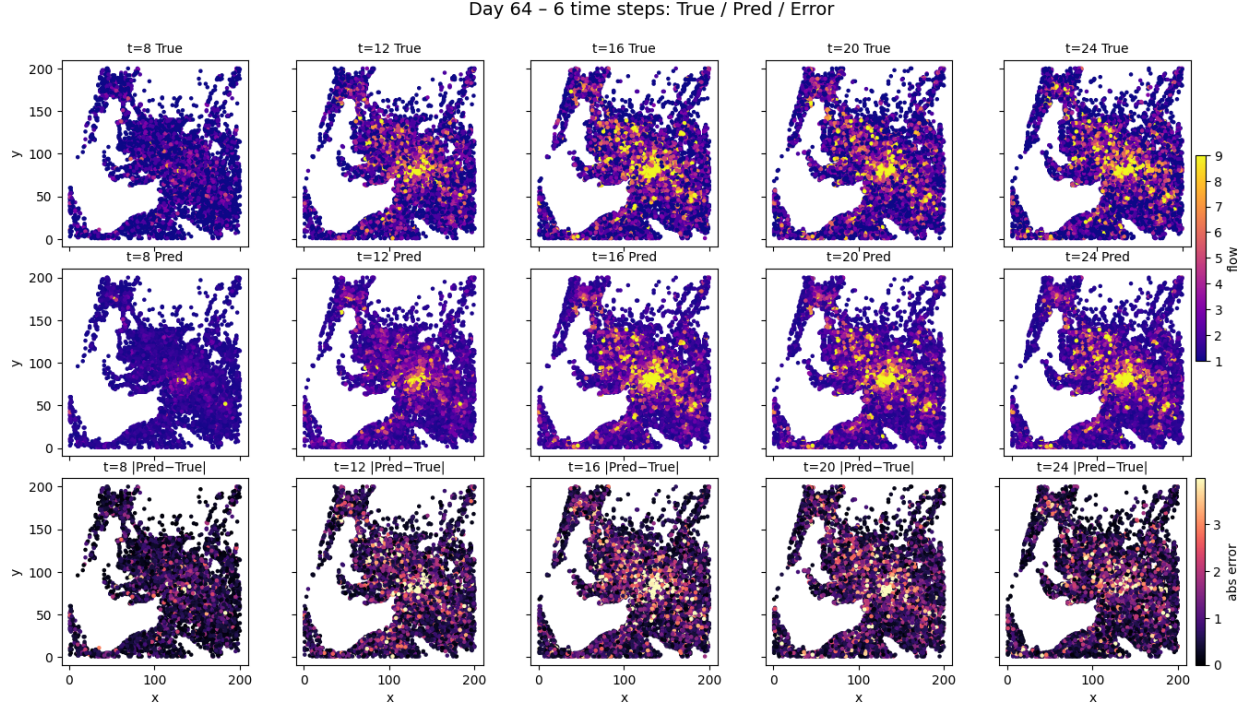


Figure 3. Multi-step spatial prediction on Day 64 across six time slots. Rows: true flow (top), LSTM prediction (middle), absolute error (bottom).

Conclusion

This project demonstrates that temporal structure is essential for grid-level mobility forecasting: a compact two-layer LSTM outperforms a non-temporal baseline by two orders of magnitude. Spatial error maps show that remaining prediction errors are geographically structured and concentrated in high-intensity urban cores, pointing toward targeted improvements via attention mechanisms, graph neural network layers for spatial propagation, and event-level covariates to better capture irregular demand spikes.

References

Yabe, T., Tsubouchi, K., Shimizu, T., Sekimoto, Y., & Urano, M. (2023). YJMob100K: City-scale and longitudinal dataset of anonymized human mobility trajectories. *Scientific Data*, 10(1), 1–10.